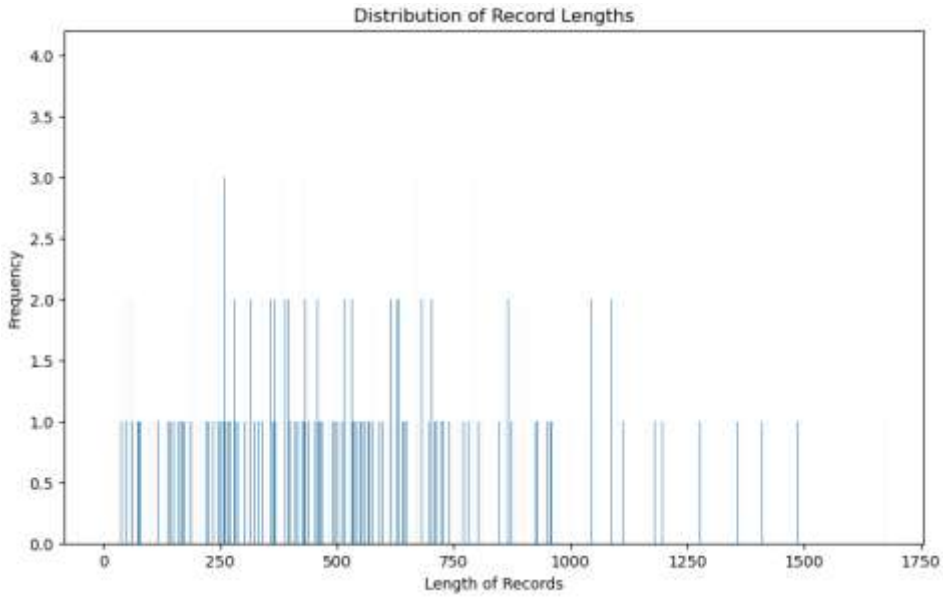
NER

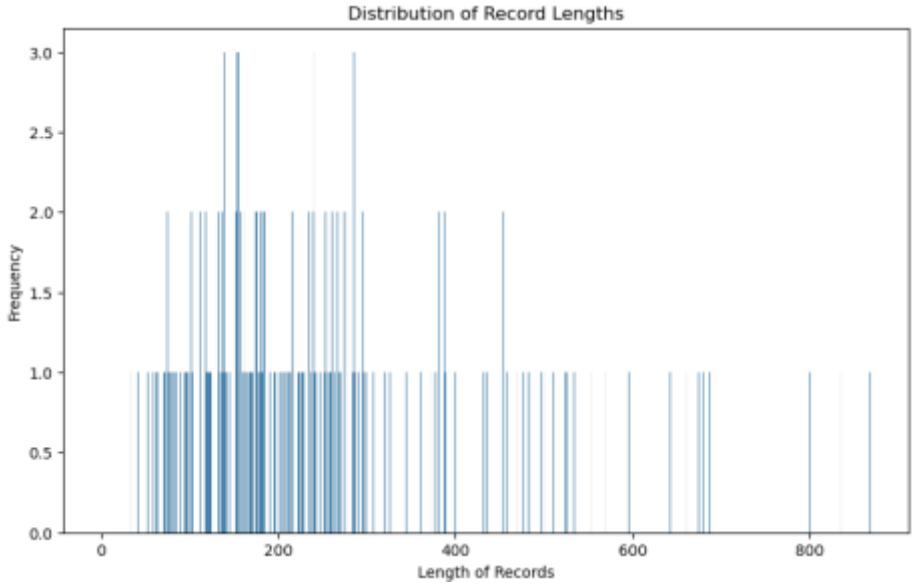
Record Length Distribution

i2b2_2010



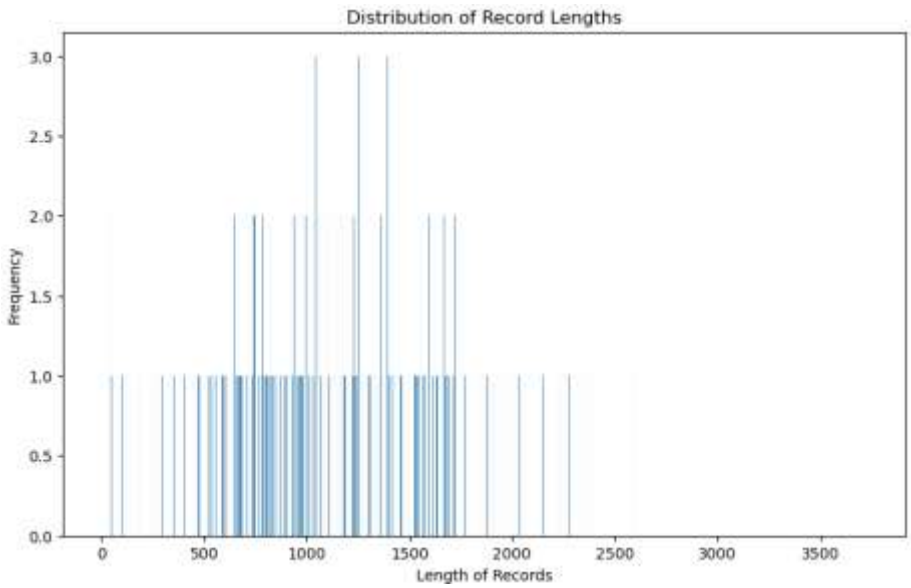
O	98143
I-problem	19581
B-problem	13110
I-treatment	8745
I-test	8737
B-treatment	8335
B-test	7188

i2b2_2012



O	18271
I-PROBLEM	6200
B-PROBLEM	3631
I-TREATMENT	3116
I-TEST	2872
B-OCCURRENCE	2821
B-TREATMENT	2464
I-OCCURRENCE	1537
I-CLINICAL_DEPT	1368
B-TEST	1269
I-DATE	1069
B-EVIDENTIAL	713
I-DURATION	532
B-DATE	388
B-DURATION	282
B-FREQUENCY	222
B-CLINICAL_DEPT	173
I-FREQUENCY	148
I-EVIDENTIAL	61
I-TIME	48
B-TIME	17

i2b2_2018



O	465570
B-Drug	23058
B-Form	11323
B-Strength	9257
B-Route	8828
B-Reason	8533
B-Frequency	7758
I-Strength	7158
I-Frequency	6022
I-Dosage	5023
B-Dosage	4825
I-Reason	3740
I-Form	2992
I-Drug	2705
B-ADE	1866
I-Duration	1323
I-ADE	883
I-Route	550
B-Duration	464

LSTM

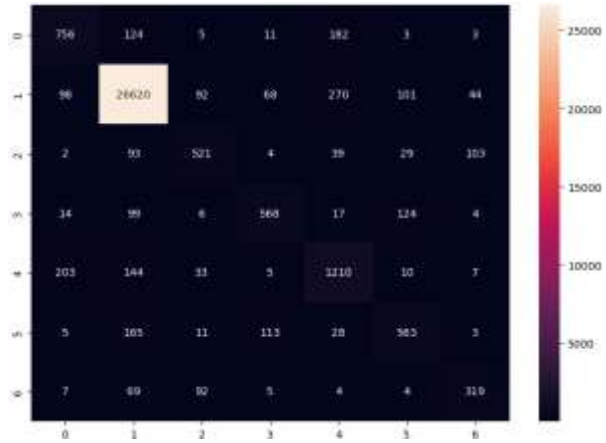
```
model = tf.keras.models.Sequential([
    Embedding(INPUT_DIM, EMBEDDING_DIM, input_length=MAX_LENGTH),
    Bidirectional(LSTM(units=LSTM_UNITS, return_sequences=True)),
    Dense(NUM_CLASSES, activation='softmax')
])

model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 1000, 216)         1924776

 bidirectional (Bidirectiona  (None, 1000, 128)        143872
 l)

 dense (Dense)               (None, 1000, 8)           1032

=================================================================
Total params: 2,069,680
Trainable params: 2,069,680
Non-trainable params: 0
_________________________________________________________________
```

i2b2_2010

MAX_LENGTH = 1000, BATCH_SIZE = 32, EPOCHS = 50

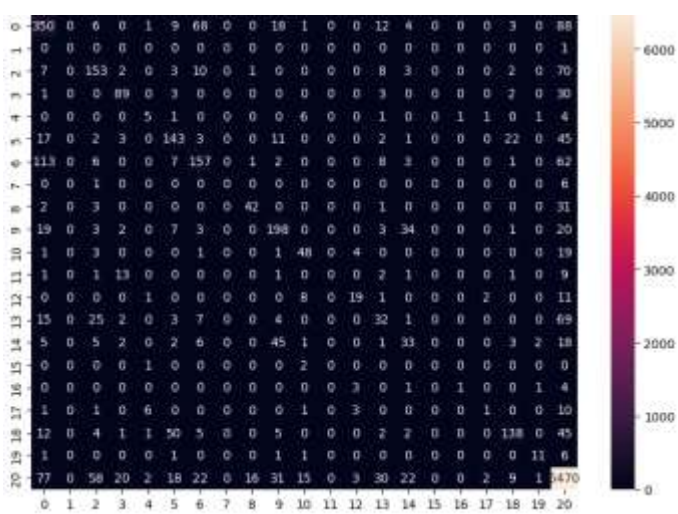
	precision	recall	f1-score	support
0	0.70	0.70	0.70	1084
1	0.97	0.98	0.97	27293
2	0.69	0.66	0.67	791
3	0.73	0.68	0.71	832
4	0.69	0.75	0.72	1612
5	0.68	0.63	0.65	888
6	0.66	0.64	0.65	500
accuracy			0.93	33000
macro avg	0.73	0.72	0.72	33000
weighted avg	0.93	0.93	0.93	33000



i2b2_2012

MAX_LENGTH = 500, BATCH_SIZE = 32, EPOCHS = 75

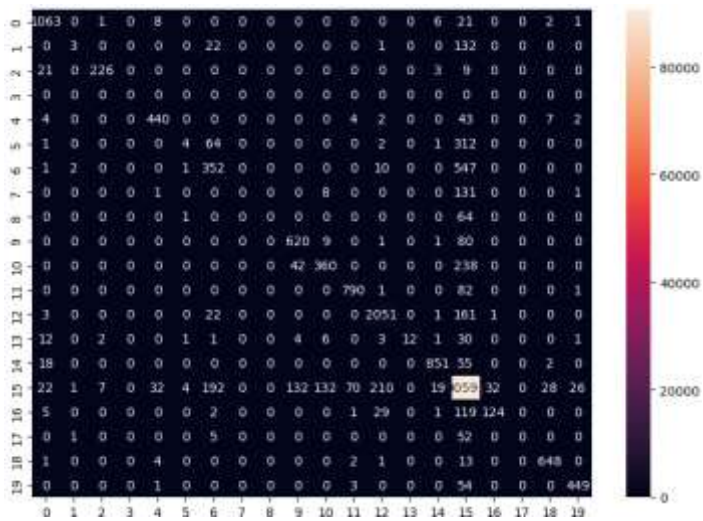
	precision	recall	f1-score	support
0	0.56	0.62	0.59	560
1	0.00	0.00	0.00	1
2	0.56	0.59	0.58	259
3	0.66	0.70	0.68	128
4	0.29	0.25	0.27	20
5	0.58	0.57	0.58	249
6	0.56	0.44	0.49	360
7	0.00	0.00	0.00	7
8	0.70	0.53	0.60	79
9	0.62	0.68	0.65	290
10	0.58	0.62	0.60	77
11	0.00	0.00	0.00	29
12	0.59	0.45	0.51	42
13	0.30	0.20	0.24	158
14	0.31	0.27	0.29	123
15	0.00	0.00	0.00	3
16	0.50	0.10	0.17	10
17	0.17	0.04	0.07	23
18	0.76	0.52	0.62	265
19	0.69	0.52	0.59	21
20	0.92	0.95	0.94	6796
accuracy			0.83	9500
macro avg	0.45	0.38	0.40	9500
weighted avg	0.82	0.83	0.82	9500



i2b2_2018

MAX_LENGTH = 2000, BATCH_SIZE = 32, EPOCHS = 30

	precision	recall	f1-score	support
0	0.92	0.96	0.94	1102
1	0.43	0.02	0.04	158
2	0.96	0.87	0.91	259
4	0.91	0.88	0.89	502
5	0.36	0.01	0.02	384
6	0.53	0.39	0.45	913
7	0.00	0.00	0.00	141
8	0.00	0.00	0.00	65
9	0.78	0.87	0.82	711
10	0.70	0.56	0.62	640
11	0.91	0.90	0.91	874
12	0.89	0.92	0.90	2239
13	1.00	0.16	0.28	73
14	0.96	0.92	0.94	926
15	0.98	0.99	0.98	91498
16	0.79	0.44	0.57	281
17	0.00	0.00	0.00	58
18	0.94	0.97	0.96	669
19	0.93	0.89	0.91	507
accuracy			0.97	102000
macro avg	0.68	0.57	0.59	102000
weighted avg	0.96	0.97	0.96	102000



<u>BiLSTM</u>

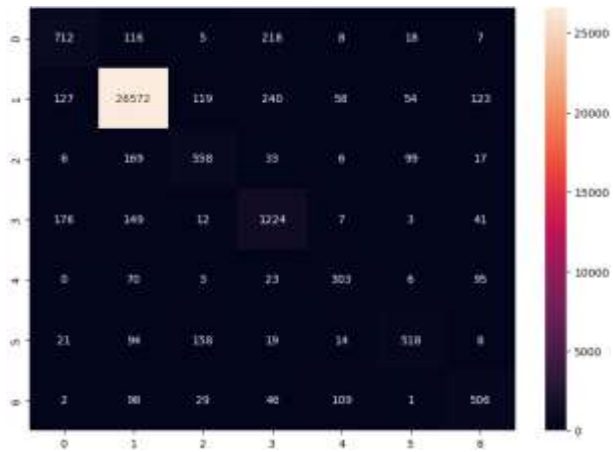
```
model = tf.keras.models.Sequential([
    Embedding(INPUT_DIM, EMBEDDING_DIM, input_length=MAX_LENGTH),
    Bidirectional(LSTM(units=LSTM1_UNITS, return_sequences=True)),
    Bidirectional(LSTM(units=LSTM2_UNITS, return_sequences=True)),
    Dense(DENSE_DIM, activation='relu'),
    Dense(NUM_CLASSES, activation='softmax')
])


model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 1000, 64)          570304

 bidirectional (Bidirectiona  (None, 1000, 128)        66048
 l)

 bidirectional_1 (Bidirectio  (None, 1000, 64)         41216
 nal)

 dense (Dense)               (None, 1000, 64)          4160

 dense_1 (Dense)             (None, 1000, 8)           520

=================================================================
Total params: 682,248
Trainable params: 682,248
Non-trainable params: 0
_________________________________________________________________
```

i2b2_2010

MAX_LENGTH = 1000, BATCH_SIZE = 32, EPOCHS = 50

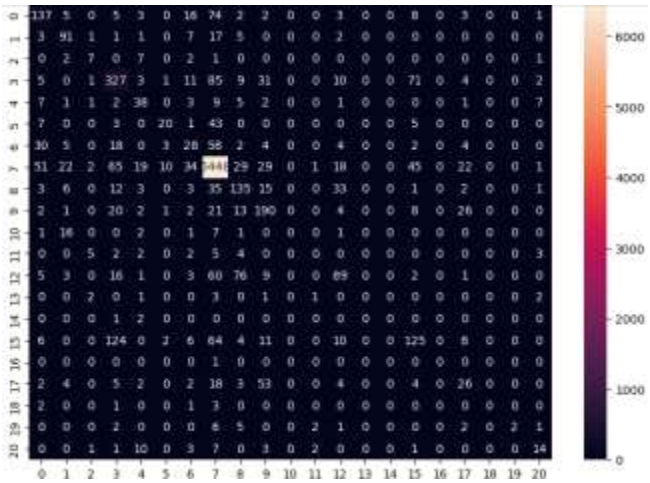
	precision	recall	f1-score	support
0	0.68	0.66	0.67	1084
1	0.97	0.97	0.97	27293
2	0.63	0.63	0.63	888
3	0.68	0.76	0.72	1612
4	0.60	0.61	0.60	500
5	0.74	0.62	0.68	832
6	0.63	0.64	0.64	791
accuracy			0.92	33000
macro avg	0.71	0.70	0.70	33000
weighted avg	0.92	0.92	0.92	33000



i2b2_2012

MAX_LENGTH = 500, BATCH_SIZE = 32, EPOCHS = 75

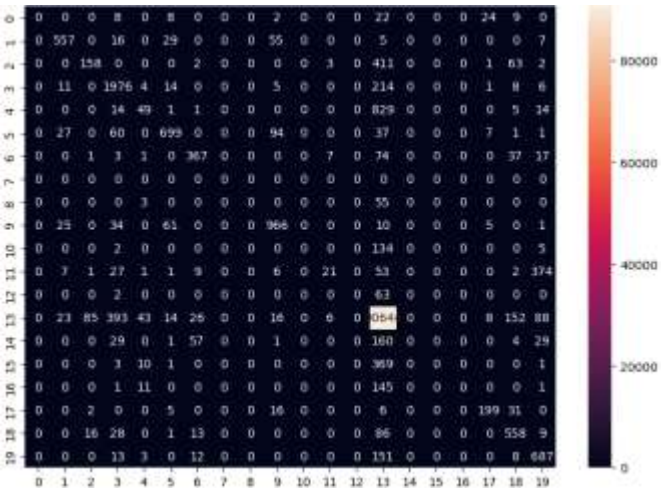
	precision	recall	f1-score	support
0	0.52	0.53	0.53	259
1	0.58	0.71	0.64	128
2	0.35	0.35	0.35	20
3	0.54	0.58	0.56	560
4	0.40	0.49	0.44	77
5	0.54	0.25	0.34	79
6	0.22	0.18	0.20	158
7	0.93	0.95	0.94	6796
8	0.46	0.54	0.50	249
9	0.54	0.66	0.59	290
10	0.00	0.00	0.00	29
11	0.00	0.00	0.00	23
12	0.49	0.34	0.40	265
13	0.00	0.00	0.00	10
14	0.00	0.00	0.00	3
15	0.46	0.35	0.40	360
16	0.00	0.00	0.00	1
17	0.26	0.21	0.23	123
18	0.00	0.00	0.00	7
19	1.00	0.10	0.17	21
20	0.42	0.33	0.37	42
accuracy			0.81	9500
macro avg	0.37	0.31	0.32	9500
weighted avg	0.80	0.81	0.80	9500



i2b2_2018

MAX_LENGTH = 2000, BATCH_SIZE = 32, EPOCHS = 30

	precision	recall	f1-score	support
0	0.00	0.00	0.00	73
1	0.86	0.83	0.84	669
2	0.60	0.25	0.35	640
3	0.76	0.88	0.82	2239
4	0.39	0.05	0.09	913
5	0.84	0.75	0.79	926
6	0.75	0.72	0.74	507
8	0.00	0.00	0.00	58
9	0.83	0.88	0.85	1102
10	0.00	0.00	0.00	141
11	0.57	0.04	0.08	502
12	0.00	0.00	0.00	65
13	0.97	0.99	0.98	91498
14	0.00	0.00	0.00	281
15	0.00	0.00	0.00	384
16	0.00	0.00	0.00	158
17	0.81	0.77	0.79	259
18	0.64	0.78	0.70	711
19	0.55	0.79	0.65	874
accuracy			0.95	102000
macro avg	0.45	0.41	0.40	102000
weighted avg	0.93	0.95	0.94	102000



BiLSTM with Time Distributed Dense Layers

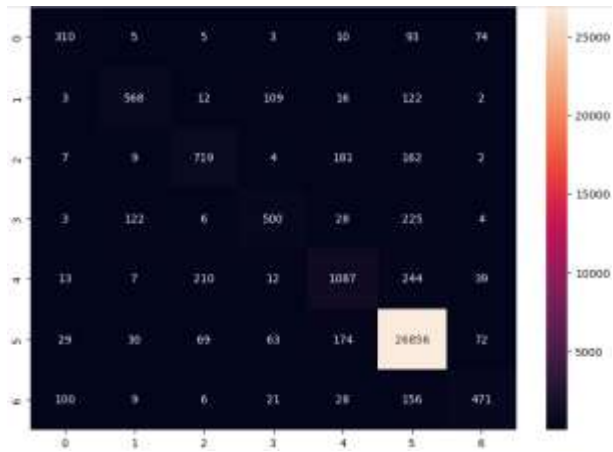
```
model = tf.keras.models.Sequential([

    # Add an embedding layer to learn the representations of words in the input text
    tf.keras.layers.Embedding(input_dim=INPUT_DIM, output_dim=EMBEDDING_DIM, input_length=MAX_LENGTH),

    # Add a Bidirectional LSTM layer to capture contextual information from both directions
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(units=LSTM_UNITS, return_sequences=True)),

    # Add a TimeDistributed dense layer to apply a dense layer to each time step of the LSTM output
    tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(units=DENSE_UNITS, activation='relu')),

    # Add a TimeDistributed dense layer to apply a dense layer to each time step of the previous dense layer output
    tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(units=NUM_CLASSES, activation='softmax'))
])


model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 1000, 64)          570304

 bidirectional (Bidirectiona  (None, 1000, 128)        66048
 l)

 time_distributed (TimeDistr  (None, 1000, 64)         8256
 ibuted)

 time_distributed_1 (TimeDis  (None, 1000, 8)          520
 tributed)

=================================================================
Total params: 645,128
Trainable params: 645,128
Non-trainable params: 0
_________________________________________________________________
```

i2b2_2010

MAX_LENGTH = 1000, BATCH_SIZE = 32, EPOCHS = 50

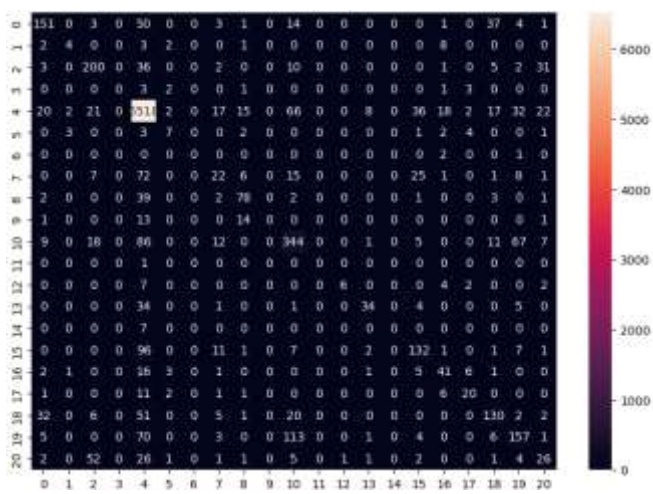
	precision	recall	f1-score	support
0	0.67	0.62	0.64	500
1	0.76	0.68	0.72	832
2	0.70	0.66	0.68	1084
3	0.70	0.56	0.63	888
4	0.71	0.67	0.69	1612
5	0.96	0.98	0.97	27293
6	0.71	0.60	0.65	791
accuracy			0.92	33000
macro avg	0.74	0.68	0.71	33000
weighted avg	0.92	0.92	0.92	33000



i2b2_2012

MAX_LENGTH = 500, BATCH_SIZE = 32, EPOCHS = 75

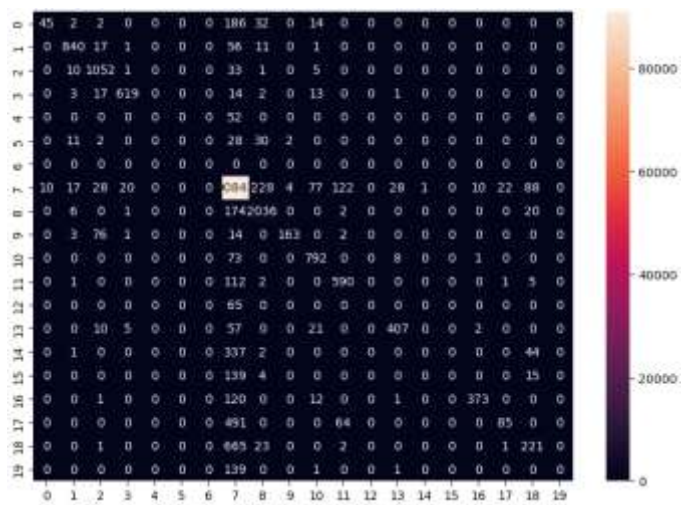
	precision	recall	f1-score	support
0	0.66	0.57	0.61	265
1	0.40	0.20	0.27	20
2	0.65	0.69	0.67	290
3	0.00	0.00	0.00	10
4	0.91	0.96	0.94	6796
5	0.37	0.30	0.33	23
6	0.00	0.00	0.00	3
7	0.27	0.14	0.18	158
8	0.64	0.61	0.62	128
9	0.00	0.00	0.00	29
10	0.58	0.61	0.59	560
11	0.00	0.00	0.00	1
12	0.86	0.29	0.43	21
13	0.71	0.43	0.54	79
14	0.00	0.00	0.00	7
15	0.61	0.51	0.56	259
16	0.48	0.53	0.50	77
17	0.54	0.48	0.51	42
18	0.61	0.52	0.56	249
19	0.54	0.44	0.48	360
20	0.27	0.21	0.24	123
accuracy			0.83	9500
macro avg	0.43	0.36	0.38	9500
weighted avg	0.81	0.83	0.82	9500



i2b2_2018

MAX_LENGTH = 2000, BATCH_SIZE = 32, EPOCHS = 30

	precision	recall	f1-score	support
0	0.82	0.16	0.27	281
1	0.94	0.91	0.92	926
2	0.87	0.95	0.91	1102
3	0.96	0.93	0.94	669
4	0.00	0.00	0.00	58
5	0.00	0.00	0.00	73
7	0.97	0.99	0.98	91498
8	0.86	0.91	0.88	2239
9	0.96	0.63	0.76	259
10	0.85	0.91	0.88	874
11	0.75	0.83	0.79	711
12	0.00	0.00	0.00	65
13	0.91	0.81	0.86	502
14	0.00	0.00	0.00	384
15	0.00	0.00	0.00	158
16	0.97	0.74	0.84	507
17	0.78	0.13	0.23	640
18	0.55	0.24	0.34	913
19	0.00	0.00	0.00	141
accuracy			0.96	102000
macro avg	0.59	0.48	0.50	102000
weighted avg	0.95	0.96	0.95	102000



BiLSTM with Time Distributed Dense Layers and CNN

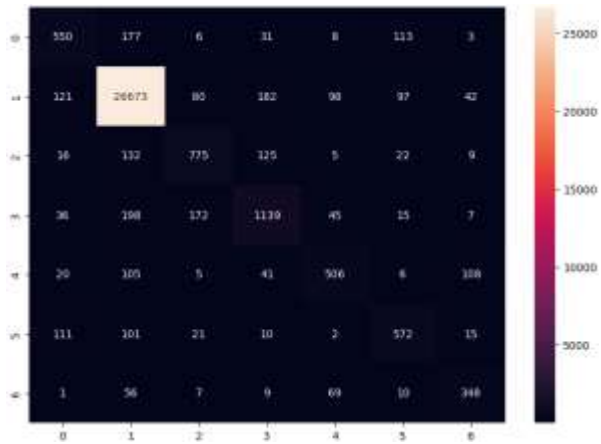
```
model = tf.keras.models.Sequential([

    # Add a word embedding layer to learn the representations of words in the input text
    tf.keras.layers.Embedding(input_dim=INPUT_DIM, output_dim=EMBEDDING_DIM, input_length=MAX_LENGTH),

    # Add a Convolutional Neural Network (CNN) layer to capture local patterns in the text
    tf.keras.layers.Conv1D(filters=32, kernel_size=3, padding='same', activation='relu'),

    # Add a Bidirectional LSTM layer to capture contextual information from both directions
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(units=LSTM_UNITS, return_sequences=True)),

    # Add a TimeDistributed dense layer to apply a dense layer to each time step of the LSTM output
    tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(units=DENSE_UNITS, activation='relu')),

    # Add a dropout layer to reduce overfitting
    tf.keras.layers.Dropout(rate=DROPOUT),

    # Add a dense layer with softmax activation to output the probabilities of each class
    tf.keras.layers.Dense(units=NUM_CLASSES, activation='softmax'),
])

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 1000, 64)          570304

 conv1d (Conv1D)             (None, 1000, 32)          6176

 bidirectional (Bidirectiona  (None, 1000, 128)        49664
 l)

 time_distributed (TimeDistr  (None, 1000, 64)         8256
 ibuted)

 dropout (Dropout)           (None, 1000, 64)          0

 dense_1 (Dense)             (None, 1000, 8)           520

=================================================================
Total params: 634,920
Trainable params: 634,920
Non-trainable params: 0
_________________________________________________________________
```

i2b2_2010

MAX_LENGTH = 1000, BATCH_SIZE = 32, EPOCHS = 50

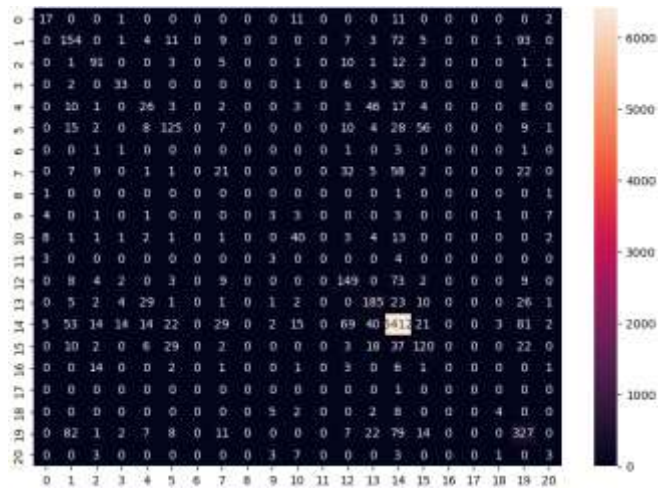
	precision	recall	f1-score	support
0	0.64	0.62	0.63	888
1	0.97	0.98	0.97	27293
2	0.73	0.71	0.72	1084
3	0.74	0.71	0.72	1612
4	0.69	0.64	0.66	791
5	0.69	0.69	0.69	832
6	0.65	0.70	0.67	500
accuracy			0.93	33000
macro avg	0.73	0.72	0.72	33000
weighted avg	0.93	0.93	0.93	33000



i2b2_2012

MAX_LENGTH = 500, BATCH_SIZE = 32, EPOCHS = 75

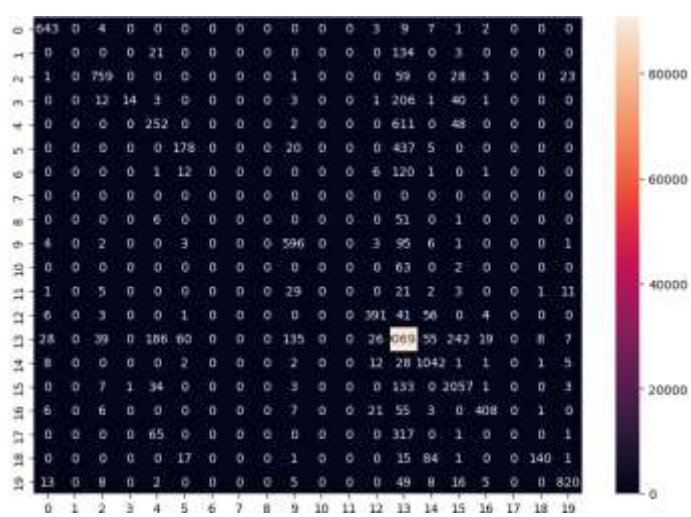
	precision	recall	f1-score	support
0	0.45	0.40	0.43	42
1	0.44	0.43	0.44	360
2	0.62	0.71	0.66	128
3	0.56	0.42	0.48	79
4	0.27	0.21	0.24	123
5	0.60	0.47	0.53	265
6	0.00	0.00	0.00	7
7	0.21	0.13	0.16	158
8	0.00	0.00	0.00	3
9	0.18	0.13	0.15	23
10	0.47	0.52	0.49	77
11	0.00	0.00	0.00	10
12	0.49	0.58	0.53	259
13	0.56	0.64	0.59	290
14	0.93	0.94	0.94	6796
15	0.51	0.48	0.49	249
16	0.00	0.00	0.00	29
17	0.00	0.00	0.00	1
18	0.40	0.19	0.26	21
19	0.54	0.58	0.56	560
20	0.14	0.15	0.15	20
accuracy			0.81	9500
macro avg	0.35	0.33	0.34	9500
weighted avg	0.80	0.81	0.81	9500



i2b2_2018

MAX_LENGTH = 2000, BATCH_SIZE = 32, EPOCHS = 30

	precision	recall	f1-score	support
0	0.91	0.96	0.93	669
1	0.00	0.00	0.00	158
2	0.90	0.87	0.88	874
3	0.93	0.05	0.09	281
4	0.44	0.28	0.34	913
5	0.65	0.28	0.39	640
6	0.00	0.00	0.00	141
8	0.00	0.00	0.00	58
9	0.74	0.84	0.79	711
10	0.00	0.00	0.00	65
11	0.00	0.00	0.00	73
12	0.84	0.78	0.81	502
13	0.97	0.99	0.98	91498
14	0.82	0.95	0.88	1102
15	0.84	0.92	0.88	2239
16	0.92	0.80	0.86	507
17	0.00	0.00	0.00	384
18	0.93	0.54	0.68	259
19	0.94	0.89	0.91	926
accuracy			0.96	102000
macro avg	0.57	0.48	0.50	102000
weighted avg	0.95	0.96	0.95	102000



Performance Metrics of the four LSTM Models on the three i2b2 Datasets

	i2b2_2010	i2b2_2012	i2b2_2018
LSTM	Accuracy: 93%	Accuracy: 83%	Accuracy: 97%
	Precision: 93%	Precision: 82%	Precision: 96%
	Recall: 93%	Recall: 83%	Recall: 97%
	f1-Score: 93%	f1-Score: 82%	f1-Score: 96%
BiLSTM	Accuracy: 92%	Accuracy: 81%	Accuracy: 95%
	Precision: 92%	Precision: 80%	Precision: 93%
	Recall: 92%	Recall: 81%	Recall: 95%
	f1-Score: 92%	f1-Score: 80%	f1-Score: 94%
BiLSTM with Time Distributed Dense Layers	Accuracy: 92%	Accuracy: 83%	Accuracy: 96%
	Precision: 92%	Precision: 81%	Precision: 95%
	Recall: 92%	Recall: 83%	Recall: 96%
	f1-Score: 92%	f1-Score: 82%	f1-Score: 95%
BiLSTM with Time Distributed Dense Layers and CNN	Accuracy: 93%	Accuracy: 81%	Accuracy: 96%
	Precision: 93%	Precision: 80%	Precision: 95%
	Recall: 93%	Recall: 81%	Recall: 96%
	f1-Score: 93%	f1-Score: 81%	f1-Score: 95%

Average Time per Epoch for the four LSTM Models on the three i2b2 Datasets

	i2b2_2010	i2b2_2012	i2b2_2018
LSTM	39.54 s	6.36 s	203.67 s
BiLSTM	10.72 s	2.97 s	43.57 s
BiLSTM with Time Distributed Dense Layers	5.62 s	1.48 s	64.60 s
BiLSTM with Time Distributed Dense Layers and CNN	6.08 s	1.76 s	64.00 s

EL

BERT

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

def get_embedding(text):
    inputs = tokenizer(text, return_tensors='pt', truncation=True, padding=True, max_length=10)
    outputs = model(**inputs)
    return outputs.last_hidden_state.mean(dim=1).detach().numpy()
```

```
from sklearn.metrics.pairwise import cosine_similarity
def find_matches(entities, entity_embeddings, second_entity_embeddings, threshold=0.85):
    matches = {}
    for entity, embedding in entity_embeddings.items():
        max_similarity = 0
        best_match = None
        for second_entity, second_embedding in second_entity_embeddings.items():
            similarity = cosine_similarity(embedding, second_embedding)[0][0]
            if similarity > max_similarity:
                max_similarity = similarity
                best_match = second_entity
        matches[entity] = (max_similarity >= threshold, best_match, max_similarity)
    return matches
```

Results:-

Embeddings Match in the three i2b2 Datasets through Different Levels of Linking

Level of Linking	i2b2_2010	i2b2_2012	i2b2_2018
Exact Match	1.06%	1.21%	1.41%
BERT	29.94%	22.61%	46.56%

Embeddings Match in the i2b2 2018 Dataset using the three Models

BERT	BioBERT	SciBERT
46.56%	99.05%	47.35%

Embeddings Match in the three Dataset using BioBERT

2010	2012	2018
98.18%	98.26%	99.05%